

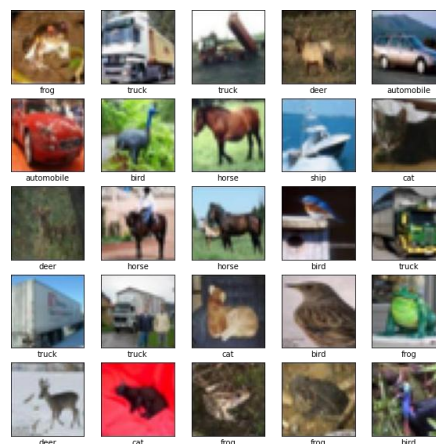


IUT de Paris — Rives de Seine

Université Paris Cité — Bachelor Universitaire de Technologie
Science des Données — BUT SD S6 — 2025–2026

Projet SAE — Apprentissage pour l'IA

Étude 2 : Classification d'images sur la base CIFAR-10



Auteurs

Chan Sing Man Nathan
Franceschin Camille
Ratsimba Manohy

Année universitaire 2025–2026

Table des matières

1. Introduction.....	3
2. Exploration des données	3
2.1 Description générale du jeu de données.....	3
2.2 Structure des données	3
2.3 Répartition des classes.....	4
3. Préparation des données.....	4
3.1 Normalisation des entrées	4
4. Modèle baseline — MLP	4
4.1 Architecture	5
4.2 Résultats et analyse.....	5
5. Modèles CNN avancés.....	5
5.1 Architecture commune.....	5
5.2 Les quatre modèles comparés	6
5.3 Filtres 3×3 versus 5×5.....	6
5.4 Data augmentation et lien avec les epochs.....	6
6. Analyse critique des résultats.....	7
6.1 Comparaison globale des modèles.....	7
6.2 Graphiques de comparaison	7
6.3 Analyse des erreurs du meilleur modèle	8
Bonus : Confusion chat / chien	9
7. Conclusion	9

1. Introduction

Le principe de notre étude repose sur un apprentissage statistique où l'ordinateur s'appuie sur des données antérieures pour apprendre à classifier, prédire ou encore effectuer des régressions sur des données futures.

Dans le cadre de la compréhension de ces réseaux de neurones, nous réaliserons ce travail dans le cadre d'un problème de classification d'images, avec pour objectif d'entraîner un modèle capable de reconnaître automatiquement la catégorie d'un objet à partir d'une image. Nous comparerons une approche de référence simple (MLP), puis des architectures plus adaptées aux images (CNN), avec et sans augmentation de données.

2. Exploration des données

2.1 Description générale du jeu de données

Pour cette étude nous utilisons le jeu de données **CIFAR-10**, une base constituée de 60 000 images couleur réparties en 10 classes distinctes. Ces images représentent ces objets : avions, automobiles, oiseaux, chats, cerfs, chiens, grenouilles, chevaux, bateaux et camions.

La classification est encodée sous forme d'entiers de 0 à 9 dans la variable de sortie. Le tableau ci-dessous détaille la correspondance entre ces entiers et les classes réelles.

Label	Classe	Label	Classe
0	Avion	5	Chien
1	Automobile	6	Grenouille
2	Oiseau	7	Cheval
3	Chat	8	Bateau
4	Cerf	9	Camion

Tableau 1 — Les 10 classes du dataset CIFAR-10

2.2 Structure des données

Chaque image possède une structure $32 \times 32 \times 3$: 32 pixels de largeur, 32 pixels de hauteur, et 3 canaux correspondant aux composantes couleur Rouge, Vert et Bleu (RGB). Les valeurs des pixels sont des entiers allant de 0 à 255, et la variable de sortie est un entier de 0 à 9 représentant la classe de l'image.

Lors du chargement de la base l'ensemble de données est divisé en deux sous-ensembles directement pour l'entraînement et la validation :

- Ensemble d'entraînement (X_{train}) : 50 000 images, soit $50\,000 \times 32 \times 32 \times 3 = 153,6$ millions de valeurs
 - Ensemble de test (X_{test}) : 10 000 images, utilisées uniquement pour l'évaluation Finale
-

Variable	Forme	Description
X_train	(50 000, 32, 32, 3)	50 000 images d'entraînement
y_train	(50 000, 1)	50 000 labels (entiers de 0 à 9)
X_test	(10 000, 32, 32, 3)	10 000 images de test
y_test	(10 000, 1)	10 000 labels de test

Tableau 2 — Structure des ensembles de données

2.3 Répartition des classes

En examinant la distribution des données, on constate que le dataset est parfaitement équilibré car chaque classe contient exactement 5 000 images en entraînement et 1 000 en test. Cette propriété est favorable à l'entraînement car elle évite tout biais lié à un déséquilibre de représentation entre classes. Les variables en entrée vont de 0 à 255 et la sortie est la classification.

3. Préparation des données

3.1 Normalisation des entrées

Avant de commencer l'étude, les données étant dans l'intervalle $[0, 255]$, ces valeurs sont trop grandes et pourraient provoquer des gradients très importants, déstabilisant l'apprentissage. Le gradient, qui lui est la mesure mathématique qui indique dans quelle direction et de quelle ampleur les poids du réseau doivent être ajustés pour réduire l'erreur ainsi si les valeurs d'entrée sont très grandes, ce gradient explose et les poids augmentent sans converger. Cette normalisation permet donc de ramener toutes les valeurs des pixels dans l'intervalle $[0, 1]$ en divisant par 255.

4. Modèle baseline — MLP

Une fois les données préparées, la suite de l'étude consiste à faire un premier modèle de base qui servira de référence pour évaluer l'apport des architectures plus complexes qu'on verra plus tard. Ce modèle de référence est un Perceptron Multi-Couches (MLP). La particularité de ce modèle est que ses couches Dense de chaque neurone est connecté à tous les neurones de la couche précédente. Pour traiter une image, il est donc nécessaire de l'aplatir, ce qu'on va faire avec (Flatten), ce qui transforme chaque image $32 \times 32 \times 3$ en un vecteur de 3 072 valeurs.

4.1 Architecture

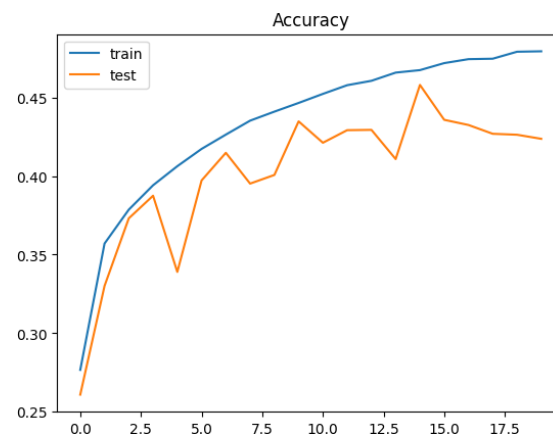
Couche	Output Shape	Paramètres
Flatten	(None, 3072)	0
Dense(32, ReLU)	(None, 32)	98 336
Dense(10, Softmax)	(None, 10)	330
Total		98 666

Tableau 3 — Architecture du modèle baseline MLP

4.2 Résultats et analyse

En regardant le résultat du modèle baseline, on peut voir que l'accuracy atteint environ 42 % sur les données de test après 20 epochs.

L'analyse des courbes nous montre un léger overfitting : la courbe d'accuracy sur les données d'entraînement progresse régulièrement, tandis que celle sur le test est instable et reste inférieure. Ce comportement s'explique par une limite fondamentale du MLP pour les images car la couche Flatten() détruit la structure spatiale 2D et le réseau ne sait plus que deux pixels voisins sont géographiquement proches, il les traite comme des variables indépendantes.



5. Modèles CNN avancés

Pour dépasser les limitations du MLP, nous construisons les réseaux de neurones convolutifs (CNN). Contrairement au MLP, les CNN préservent la structure spatiale 2D des images, donc résous le problème qu'on vient juste de voir. Leur fonctionnement repose sur des filtres qui glissent sur l'image pour détecter des motifs (bords, textures, formes) peu importe leur position dans l'image. On a ainsi développé 4 modèles CNN, 2 sans data augmentation avec fenêtre de 3x3 et 5x5 chacun.

5.1 Architecture commune

Tous les CNN développés partagent la même structure de base : deux blocs de convolution suivis de couches de classification Dense. Chaque bloc est composé d'une couche Conv2D avec activation ReLU, suivie d'un MaxPooling2D qui divise la résolution spatiale par 2 en ne conservant que la valeur maximale dans chaque zone de 2x2 pixels. L'optimiseur Adam est utilisé pour tous les CNN, car il adapte automatiquement le taux d'apprentissage pour

chaque paramètre contrairement au SGD qui lui a un taux fixe pour le baseline.

5.2 Les quatre modèles comparés

Modèle	Filtres	Data Aug.
CNN_3x3	3×3	Non
CNN_5x5	5×5	Non
CNN_3x3_augmente	3×3	Oui
CNN_5x5_augmente	5×5	Oui

Tableau 4 — Comparaison des modèles CNN

5.3 Filtres 3×3 versus 5×5

La comparaison entre filtres de taille 3×3 et 5×5 révèle que les filtres 5×5 obtiennent de meilleurs résultats sur CIFAR-10. Les images ne faisant que 32×32 pixels, les objets occupent une proportion significative de l'image et un filtre plus grand capture des motifs plus larges en une seule opération, ce qui est avantageux à si basse résolution.

5.4 Data augmentation et lien avec les epochs

Ensuite on a ajouté une data augmentation qui consiste à créer des variantes artificielles de chaque image (retournements horizontaux, décalages, zooms légers) implémentées directement comme couches dans le modèle via RandomFlip, RandomTranslation, et s'activent uniquement pendant `model.fit()`.

Ainsi on constate que sur 20 epochs, les modèles sans augmentation surpassent ceux avec augmentation. Cela est dû au fait que la data augmentation rend l'apprentissage plus difficile car le réseau voit des images différentes à chaque epoch, et 20 itérations ne suffisent pas pour que les données aient eu le temps de bien s'entraîner avec les variantes. En revanche, si l'on regarde bien les courbes sur 100 epochs, le CNN 5×5 avec augmentation finit par dépasser le CNN 5×5 normal car il a plus de temps d'apprentissage. Le CNN 5×5 normal est meilleur si l'epoch est petit car les données n'ont pas eu le temps de bien s'entraîner mais si c'est grand, le CNN 5×5 augmenté a de meilleurs résultats.

6. Analyse critique des résultats

6.1 Comparaison globale des modèles

Modèle	Accuracy train	Accuracy test
Baseline MLP	0.5055	0.4587
CNN1 (3×3, sans aug.)	0.8410	0.7081
CNN2 (5×5, sans aug.)	0.8654	0.6923
CNN3 (3×3, avec aug.)	0.7814	0.7406
CNN4 (5×5, avec aug.)	0.7885	0.7494

Tableau 5 — Récapitulatif des performances de tous les modèles

6.2 Graphiques de comparaison

Les figures suivantes présentent l'évolution de l'accuracy, de la loss et de la MAE de chaque modèle au cours de l'entraînement sur 100 epochs en utilisant de l'Early Stopping avec une patience de 20 afin de ne pas avoir d'arrêt trop rapide, sur les ensembles d'entraînement (trait plein) et de test (trait pointillé).

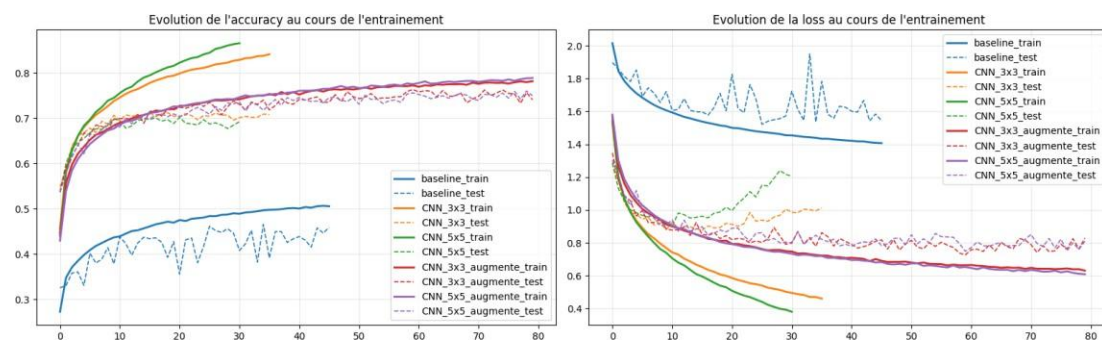


Figure 1 — Évolution de l'accuracy (gauche) et de la loss (droite) au cours de l'entraînement

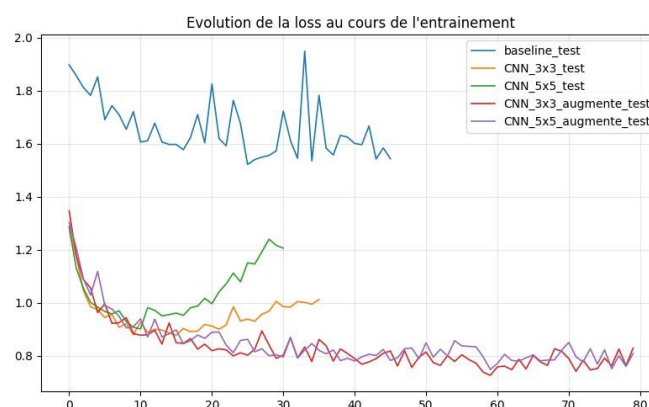


Figure 2 — Évolution de la loss sur les données de test (zoom sur les CNN)

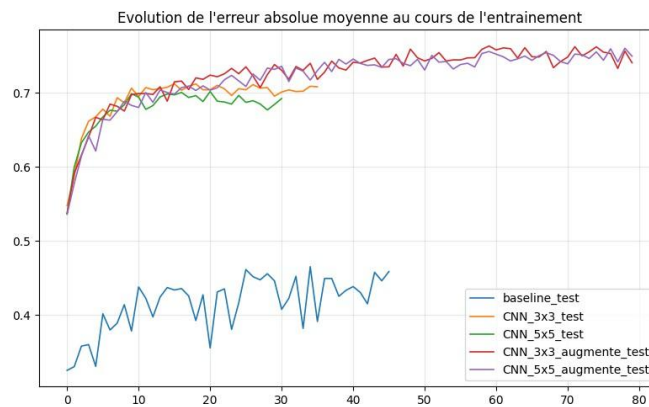


Figure 3 — Évolution de l'erreur absolue moyenne (MAE) sur les données de test

On observe clairement que les CNN (courbes orange, verte, rouge, violette) convergent bien plus rapidement et atteignent des performances nettement supérieures au baseline (courbe bleue). Sur la figure 1, le **CNN 5×5** (courbe verte, trait plein) monte très rapidement dans les 30 premières epochs, illustrant concrètement la différence entre un entraînement court et long mentionnée précédemment.

6.3 Analyse des erreurs du meilleur modèle

Le meilleur modèle **CNN 5x5 augmenté** commet 2 442 erreurs sur 10 000 images, soit une accuracy de **75,58 %**. L'analyse par classe révèle des disparités importantes : certaines classes sont beaucoup plus difficiles que d'autres.

Classe	Erreurs / 1000	Taux d'erreur
Chat	496	49,6 %
Oiseau	466	46,6 %
Chien	418	41,8 %
Cerf	323	32,3 %
Avion	286	28,6 %
Cheval	244	24,4 %
Camion	226	22,6 %
Grenouille	216	21,6 %
Automobile	196	19,6 %
Bateau	188	18,8 %

Tableau 6 — Taux d'erreur par classe, trié par ordre décroissant

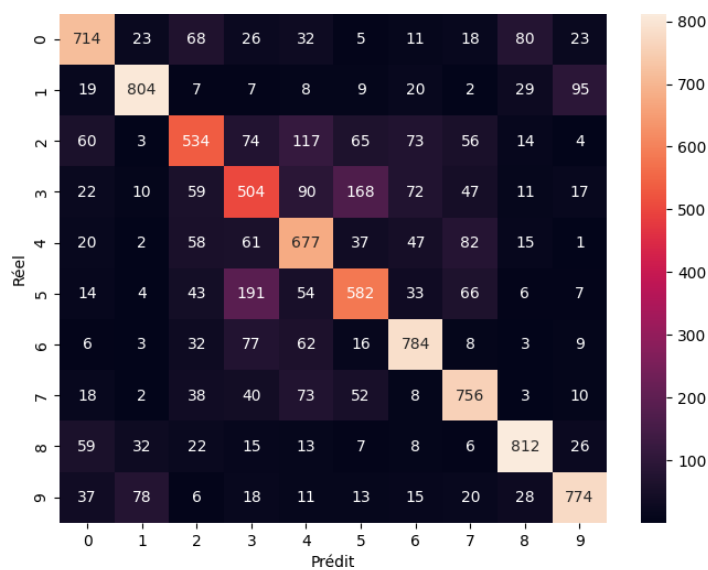
Bonus : Confusion chat / chien

Figure 7 — Matrice de confusion

L'analyse de la matrice de confusion révèle que les confusions les plus fréquentes sont entre le chat (classe 3) et le chien (classe 5). Les cases (3, 5) et (5, 3) de la matrice sont les plus chargées en dehors de la diagonale principale. Cette confusion représente la principale source d'erreur du modèle. Chat et chien partagent des silhouettes très similaires, 4 pattes, fourrure, oreilles, et la basse résolution des images ne permet pas de distinguer les détails fins. Ainsi le modèle **CNN 5x5 augmenté** classe généralement bien les images, en dehors de cette confusion entre les chiens et les chats qui représente le triple d'erreurs par rapport aux autres croisements d'erreurs.

Voici un exemple de 5 images mal classées entre chiens et chats. On comprend que le modèle à eu du mal à différencier les deux animaux étant déjà semblable pour l'être humain, mais en ajoutant la qualité de l'image cela n'est pas aberrant d'avoir de tels erreurs.



Figure 8 — Exemple de 5 images mal classées entre chats et chiens

7. Conclusion

Cette étude a permis de comparer de manière progressive différentes approches de classification d'images sur CIFAR-10. Le modèle MLP baseline établit une référence à **42 %** d'accuracy.

Les filtres 5×5 se montrent légèrement supérieurs aux filtres 3×3 sur ce dataset. La data augmentation se révèle bénéfique uniquement avec un nombre d'epochs suffisant. On constate que sur 20 epochs, c'est le **CNN 5×5** qui domine mais sur 100 epochs, le **CNN 5×5 augmenté** rend l'avantage. Les principales confusions concernent les paires chat/chien, illustration des

limites intrinsèques d'une résolution de 32×32 pixels.

L'objectif principal de ce projet nous a permis une compréhension des concepts de Deep Learning et la comparaison des différents modèles.